
REDES NEURONALES

Fernanda Sobrino

2026

Repaso Redes Neuronales Superficiales

Podemos pensar en una red neuronal como

- ▶ Una función lineal por partes
- ▶ Por ejemplo:

$$y = \phi_0 + \phi_1 a(\theta_{10} + \theta_{11}x) + \phi_2 a(\theta_{20} + \theta_{21}x) + \phi_3 a(\theta_{30} + \theta_{31}x)$$

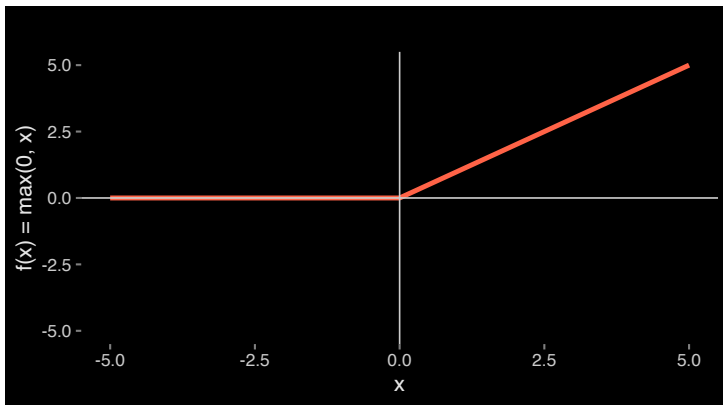
- ▶ Esta función mapea $x \rightarrow y$ y la podemos separar en tres pedazos lineales:
 - ▷ $\theta_{10} + \theta_{11}x$
 - ▷ $\theta_{20} + \theta_{21}x$
 - ▷ $\theta_{30} + \theta_{31}x$

Podemos pensar en una red neuronal como

- ▶ Después pasamos estos resultados por la función de activación $a()$ y calculamos una combinación lineal de estas usando ϕ_1 , ϕ_2 , ϕ_3 y ϕ_0
- ▶ Recordemos que podemos usar distintas funciones de activación como por ejemplo ReLU:

$$a(z) = ReLU(z) = \begin{cases} 0 & z < 0 \\ z & z \geq 0 \end{cases}$$

ReLU



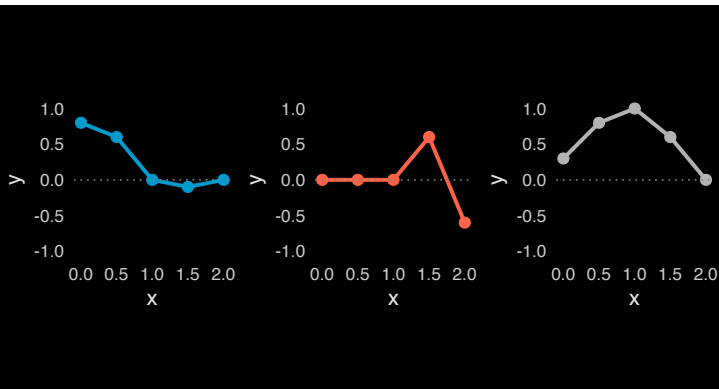
Red neuronal superficial

$$y = \phi_0 + \phi_1 a(\theta_{10} + \theta_{11}x) + \phi_2 a(\theta_{20} + \theta_{21}x) + \phi_3 a(\theta_{30} + \theta_{31}x)$$

- ▶ Modelo con 10 parámetros $\phi = \{\phi_0, \phi_1, \phi_2, \phi_3, \theta_{10}, \theta_{11}, \theta_{20}, \theta_{21}, \theta_{30}, \theta_{31}\}$
- ▶ Representa una familia de funciones
- ▶ Los parámetros van a determinar la función en particular
- ▶ Dados los parámetros podemos correr el modelo
- ▶ Dado un conjunto de entrenamiento $\{x_i, y_i\}_{i=1}^I$
- ▶ Definimos la función de pérdida $L(\phi)$
- ▶ Cambiamos los parámetros para minimizar la función de pérdida

Tres funciones distintas de esta familia

- ▶ Familia de funciones definida por la ecuación anterior con tres distintos sets de los 10 parámetros
- ▶ La relación entre x y y es lineal por partes pero la posición de las articulaciones es distinta y las pendientes



¿Por qué se ven así estas gráficas?

- ▶ Nuestra ecuación representa una familia de funciones continuas lineales por partes con hasta 4 regiones
- ▶ Cada región es un segmento de línea: ¿porqué?

$$y = \phi_0 + \phi_1 a(\theta_{10} + \theta_{11}x) + \phi_2 a(\theta_{20} + \theta_{21}x) + \phi_3 a(\theta_{30} + \theta_{31}x)$$

Neuronas/nodos:

- Podemos re-escribir nuestra ecuación de la siguiente manera:

$$y = \phi_0 + \phi_1 h_1 + \phi_2 h_2 + \phi_3 h_3$$

- Donde las neuronas/nodos (hidden units) están dadas por:

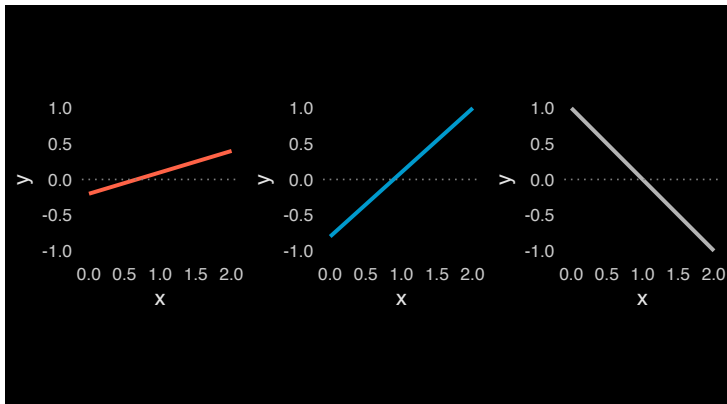
- ▷ $h_1 = a(\theta_{10} + \theta_{11}x)$

- ▷ $h_2 = a(\theta_{20} + \theta_{21}x)$

- ▷ $h_3 = a(\theta_{30} + \theta_{31}x)$

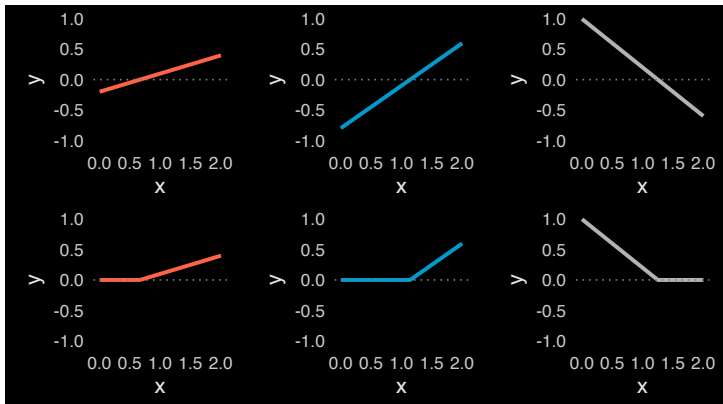
¿Cómo funciona la red neuronal?

► **Paso 1:** calcula tres funciones lineales



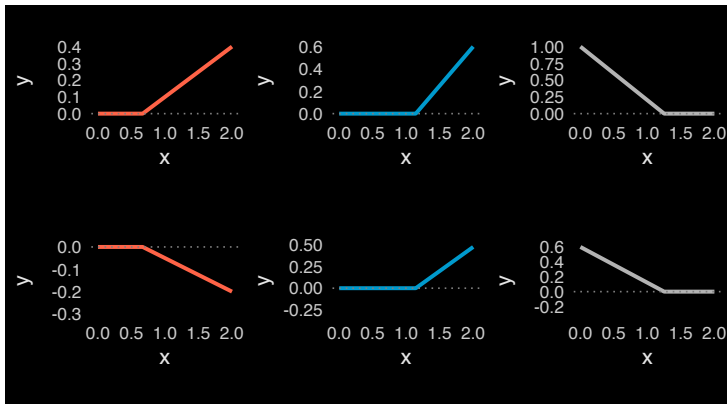
¿Cómo funciona la red neuronal?

► **Paso 2:** aplicar ReLU para crear cada neurona



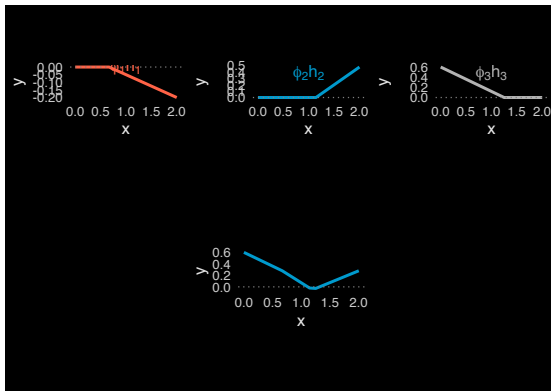
¿Cómo funciona la red neuronal?

► **Paso 3:** multiplicamos las neuronas por sus ϕ s



¿Cómo funciona la red neuronal?

- **Paso 4:** suma ponderada de las unidades ocultas para crear y
- $y = \phi_0 + \phi_1 h_1 + \phi_2 h_2 + \phi_3 h_3$



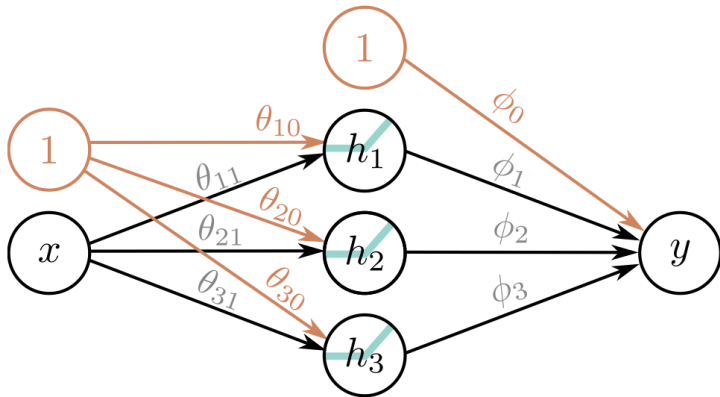
Red Neuronal:

- ▶ Cada neurona/nodo (hidden unit) contribuye con una articulación/codo a la función
- ▶ Cada neurona activa una región diferente dependiendo de si su salida es cero o no creando cuando mucho 3 articulaciones
- ▶ Estas tres articulaciones crean hasta 4 regiones (pedazos de linea)
- ▶ Tres de estas regiones son independientes, la cuarta es la suma de todas las pendientes
- ▶ Mientras que las primeras tres regiones vienen de combinaciones de neuronas activas e inactivas, la cuarta región es cuando todas las neuronas están activas

¿Qué red neuronal representa esto que acabamos de escribir?

- ▶ Tenemos 1 solo input de 1 dimensión x
- ▶ Tenemos una sola capa con 3 nodos/neuronas
- ▶ Cada nodo es una función lineal + su activación
- ▶ Tenemos un solo output que es la combinación de las activaciones de los nodos intermedios

¿Qué red neuronal representa esto que acabamos de escribir?



Teorema de Aproximación universal

- Podemos agregar tantas neuronas/nodos como queramos a una red superficial

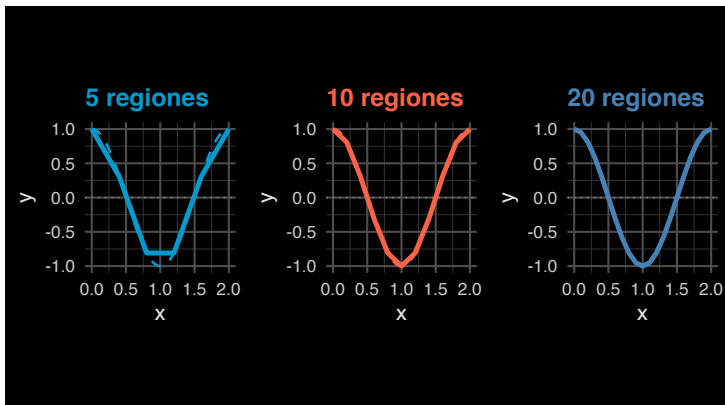
$$y = \phi_0 + \sum_{d=1}^D \phi_d h_d$$

- Donde $h_d = a(\theta_{d0} + \theta_{d1}x)$
- El número de neuronas de una red superficial mide la capacidad de la red

Teorema de Aproximación universal

- ▶ Si usamos la activación ReLU el output de una red superficial con D neuronas tiene cuando mucho D articulaciones
- ▶ Será una función lineal por partes de cuando mucho $D + 1$ regiones lineales
- ▶ Si seguimos agregando unidades ocultas podemos aproximar funciones muy complejas
- ▶ **Teorema de Aproximación Universal:** con suficientes neuronas, una red superficial puede describir cualquier función continua en un subconjunto compacto de $\mathbb{R}^D$ con precisión arbitraria

Teorema de Aproximación universal

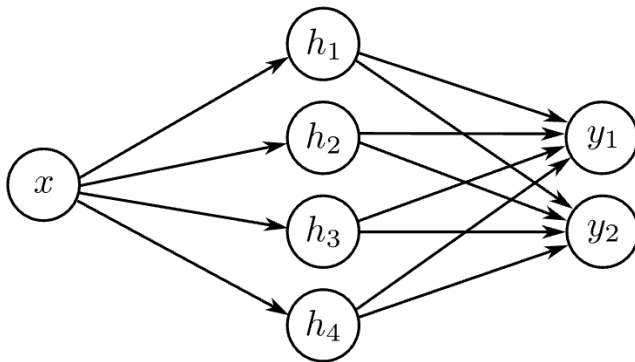


Inputs/Outputs de más dimensiones

- ▶ Todo lo que vimos se generaliza independientemente de la dimensión tanto de los inputs como de los outputs

Outputs » 1D

- Esta red tiene 1 input, 4 neuronas y dos outputs



Outputs » 1D

► ¿Cómo escribiríamos la red de la slide anterior?

► Unidades ocultas:

▷ $h_1 = a(\theta_{10} + \theta_{11}x)$

▷ $h_2 = a(\theta_{20} + \theta_{21}x)$

▷ $h_3 = a(\theta_{30} + \theta_{31}x)$

▷ $h_4 = a(\theta_{40} + \theta_{41}x)$

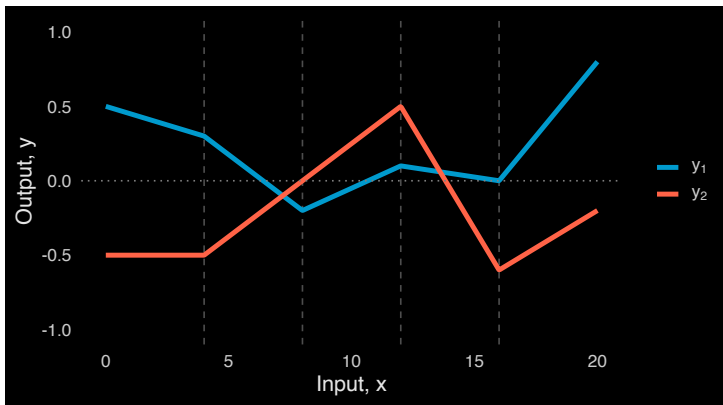
► Output:

▷ $y_1 = \phi_{10} + \phi_{11}h_1 + \phi_{12}h_2 + \phi_{13}h_3 + \phi_{14}h_4$

▷ $y_2 = \phi_{20} + \phi_{21}h_1 + \phi_{22}h_2 + \phi_{23}h_3 + \phi_{24}h_4$

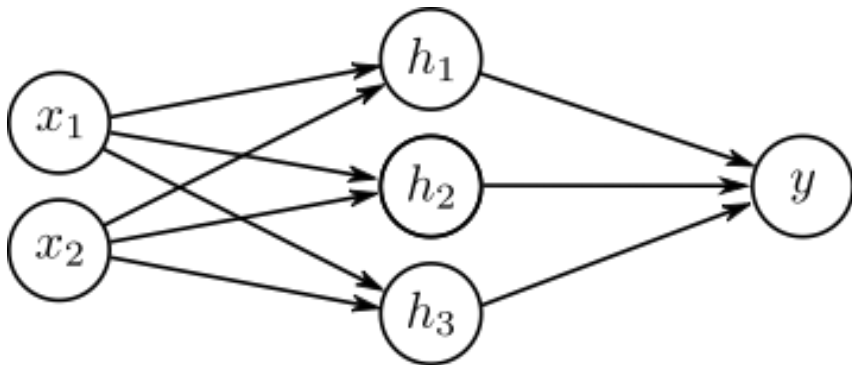
Outputs » 1D

- ¿Qué genera esta red?
- 2 funciones por partes para cada uno de los outputs



Inputs » 1D

- Esta red tiene inputs de dos dimensiones, 3 neuronas y output de 1 dimensión



Inputs » 1D

► ¿Cómo escribiríamos la red de la slide anterior?

► Neuronas:

▷ $h_1 = a(\theta_{10} + \theta_{11}x_1 + \theta_{12}x_2)$

▷ $h_2 = a(\theta_{20} + \theta_{21}x_1 + \theta_{22}x_2)$

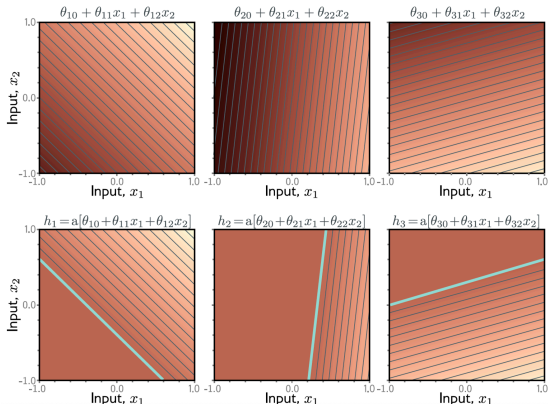
▷ $h_3 = a(\theta_{30} + \theta_{31}x_1 + \theta_{23}x_2)$

► Output:

▷ $y = \phi_0 + \phi_1h_1 + \phi_2h_2 + \phi_3h_3$

Inputs » 1D: gráficamente

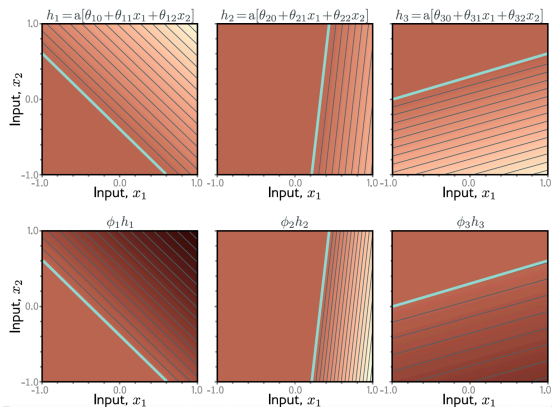
- El input de cada unidad oculta es una función lineal de x_1 y x_2 , osea un plano orientado. Las líneas son curvas de nivel
- A cada plano le aplicamos una función ReLU



Inputs » 1D: gráficamente

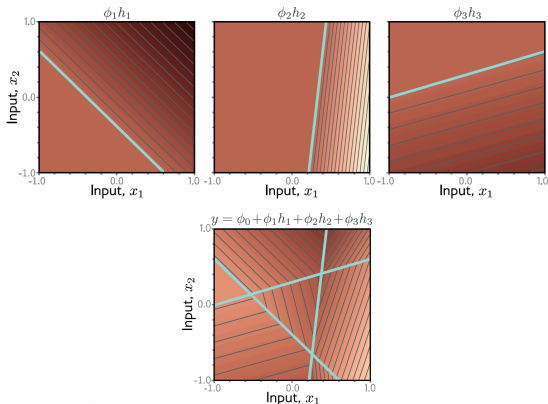
- ▶ Les aplicamos a las neuronas los pesos
- ▶ Ojo antes esto cambiaba la pendiente, pero como estamos en 2D cambia el gradiente lo que hace que las curvas de nivel se vean más o menos juntas
- ▶ ϕ escala la magnitud del gradiente (qué tan “empinado” es el plano) pero no su dirección (el ángulo del gradiente), porque multiplicas por un escalar

Inputs » 1D: gráficamente



Inputs » 1D: gráficamente

- Ponderamos los planos recortados
- El resultado es una superficie compuesta por regiones poligonales lineales convexas por partes

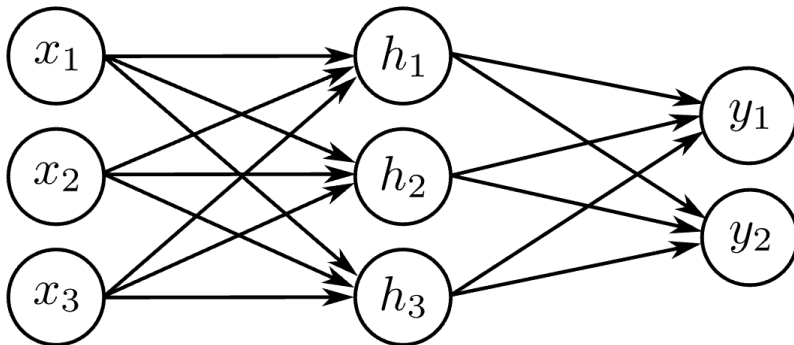


Caso General

- ▶ Podemos generalizar esto para una cantidad arbitraria de inputs/outputs y neuronas
- ▶ D_o outputs, D_i inputs y D neuronas
- ▶ $h_d = a\left(\theta_{d0} + \sum_{i=1}^{D_i} \theta_{di} x_i\right)$
- ▶ $y_j = \phi_{j0} + \sum_{d=1}^D \phi_{jd} h_d$

Caso general

► ¿Cuántos parámetros va a tener una red que se ve así?

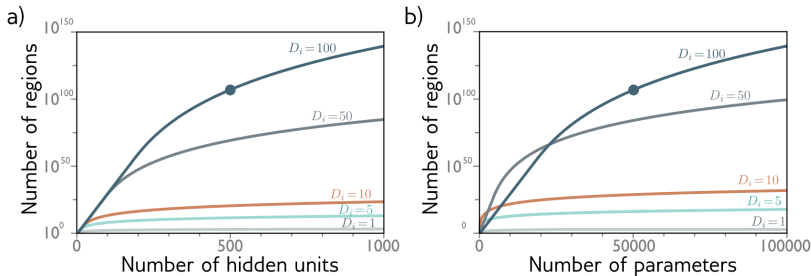


Número de regiones

- ▶ En general cada output va a consistir en politopos convexos (poliedro en espacio de dimensión superior a tres) de D dimensiones
- ▶ En el ejemplo de antes vimos que con dos inputs y tres outputs hay 7 polígonos

Número de regiones

- ▶ Con $D = 500$ y un input de dimensión 100 puede haber más de 10^{107} regiones
- ▶ Con $D = 500$ y input de dimensión 100 tenemos 51001 parámetros (estos son pocos comparados con la mayoría de los modelos actuales)



Número de regiones

- ▶ Con una red superficial con $D_i \geq 2$ inputs y D neuronas
- ▶ La cantidad de regiones lineales está dada por la intersección de los D hiperplanos creados por las articulaciones de las funciones de activación
- ▶ Zaslavsky (1975) demostró el número de regiones creadas por D hiperplanos en el espacio de entrada de dimensión $D_i \leq D$
- ▶ La cantidad de regiones creadas por D hiperplanos donde $D_i \leq D$ es:

$$2^{D_i} \leq \sum_{j=0}^{D_i} \binom{D}{j} \leq 2^D$$

Número de regiones

- ▶ 1D input con una neurona crea dos regiones (una articulación)
- ▶ 2D input con dos neuronas crea 4 regiones (dos líneas)
- ▶ 3D input con tres neuronas crea 8 regiones (3 planos)

Redes Neuronales Profundas

¿Qué es una red neuronal profunda?

- ▶ Una red con más de una capa intermedia (**hidden layer**).
- ▶ Recordemos que, si aumentamos la cantidad de neuronas/nodos en una red superficial, ganamos capacidad para describir patrones complejos.
- ▶ **Problema:** para ciertas funciones, la cantidad de neuronas necesarias en una sola capa es prohibitivamente grande.
- ▶ Las redes profundas pueden generar muchas más regiones lineales que una superficial, para un número equivalente de parámetros.
- ▶ Son más eficientes que “alargar/ensanchar” una única capa, pero su comportamiento es menos intuitivo.

Pegando dos redes superficiales

► Red 1: input para la segunda red

▷ $h_1 = a(\theta_{10} + \theta_{11}x)$

▷ $h_2 = a(\theta_{20} + \theta_{21}x)$

▷ $h_3 = a(\theta_{30} + \theta_{31}x)$

▷ $y = \phi_0 + \phi_1 h_1 + \phi_2 h_2 + \phi_3 h_3$

► Red 2: toma como input esta y y regresa y' pasando por una segunda red

▷ $h'_1 = a(\theta'_{10} + \theta'_{11}y)$

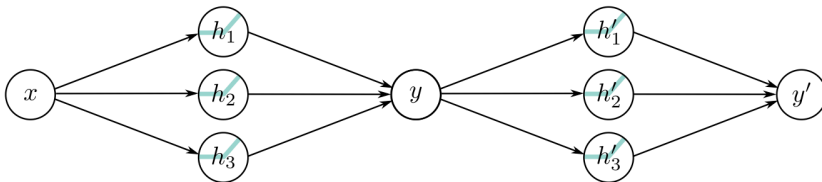
▷ $h'_2 = a(\theta'_{20} + \theta'_{21}y)$

▷ $h'_3 = a(\theta'_{30} + \theta'_{31}y)$

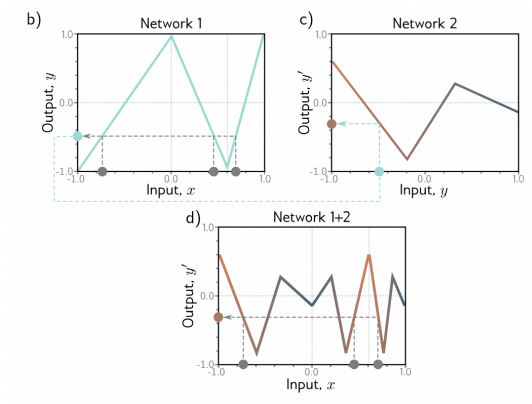
▷ $y' = \phi'_0 + \phi'_1 h'_1 + \phi'_2 h'_2 + \phi'_3 h'_3$

Pegando dos redes superficiales

- Al componerlas, obtenemos **9 regiones** lineales con sólo 20 parámetros, frente a las 7 regiones de una red superficial equivalente.



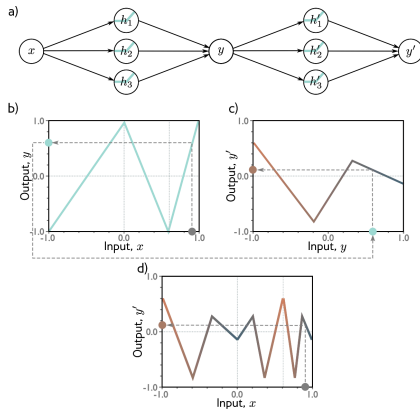
¿Cómo se ve esto?



¿Qué esta pasando acá?

- ▶ La primera red mapea inputs $x \in [-1, 1]$ a outputs $y \in [-1, 1]$ usando una función por partes con 3 partes lineales
- ▶ Varios inputs x 's (circulos grises) van a mapear a la misma y
- ▶ La segunda red es otra función lineal por partes con 3 regiones lineales, que toma y y regresa y'
- ▶ El resultado lo vemos en el último panel donde tenemos una función lineal por partes pero con 9 regiones!
- ▶ En esta última gráfica vemos como nuestras x en gris se mapean al mismo y'

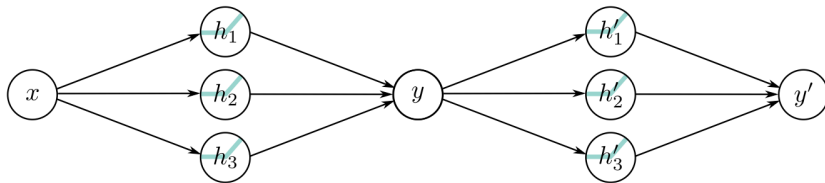
Ejemplo de como se ve esto:



Comparación:

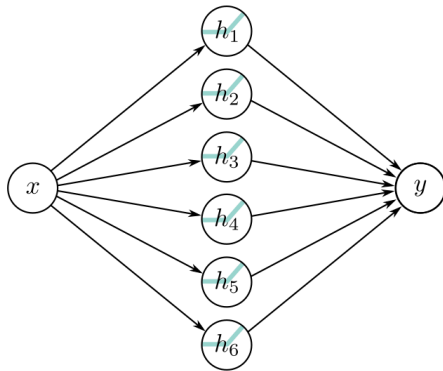
¿Cuántos parámetros y cuántas regiones produce esta red?

- ▶ Cada capa oculta tiene 3 neuronas ReLU, así que puede dividir una región lineal en hasta $3 + 1 = 4$ regiones.
- ▶ Como la segunda capa puede subdividir cada una de las 4 regiones creadas por la primera, el máximo es: $(3 + 1)(3 + 1) = 16$ regiones lineales.



Comparación:

► ¿Cuántos parámetros y cuántas regiones produce esta red?



¿Cómo se ven estas dos redes en una sola ecuación?

- ▶ Recordemos que podemos escribir las unidades ocultas de la segunda red de esta manera:
 - ▷ $h'_1 = a(\theta'_{10} + \theta'_{11}y)$
 - ▷ $h'_1 = a(\theta'_{10} + \theta'_{11}\phi_0 + \theta'_{11}\phi_1h_1 + \theta'_{11}\phi_2h_2 + \theta'_{11}\phi_3h_3)$
 - ▷ Podemos renombrar estos parámetros y escribirlo así:
 - ▷ $h'_1 = a(\psi_{10} + \psi_{11}h_1 + \psi_{12}h_2 + \psi_{13}h_3)$
- ▶ Siguiendo la lógica anterior podemos escribir a y' en términos de las x 's y todos los parámetros re-definidos

¿Cómo construye una familia de funciones esta red?

1. Las 3 neuronas h_1 , h_2 y h_3 son lineales y les aplicamos la función ReLU.
2. La pre-activación en la segunda capa se calculan tomando 3 nuevas funciones lineales. En este punto tenemos una red superficial con 3 outputs, hemos calculado 3 funciones por piezas con articulaciones entre las regiones en los mismos lugares.
3. Le aplicamos ReLU a esta segunda capa lo que junta y agrega las nuevas articulaciones a la función
4. El resultado es una combinación lineal de estas unidades ocultas

Hiperparámetros

- ▶ ¿Qué es un hiperparámetro?
- ▶ En una red neuronal profunda tenemos:
 - ▷ K capas: que tan profunda es la red
 - ▷ D_k para $k = 1, \dots, K$ neuronas por capa: que tan ancha/larga es la red
- ▶ Los seleccionamos antes de entrenar el modelo
- ▶ Para un set de hiperparámetros el modelo describe una familia de funciones y de parámetros que determinan el modelo en particular

¿Qué es un hiperparámetro?

- ▶ Podemos re-entrenar con otros parámetros... recordemos: optimización de hiperparámetros o búsqueda de hiperparámetros.
- ▶ Tomando en cuenta los hiperparámetros la red representa una familia de familias que relacionan los inputs con los outputs

¿Cómo escribimos una red neuronal profunda general?

- ▶ Para cada capa h :
 - ▷ $h_1 = a(\theta_{10} + \theta_{11}x)$
 - ▷ $h_2 = a(\theta_{20} + \theta_{21}x)$
 - ▷ $h_3 = a(\theta_{30} + \theta_{31}x)$
- ▶ Se puede re-escribir como:

$$\begin{bmatrix} h_1 \\ h_2 \\ h_3 \end{bmatrix} = a \left(\begin{bmatrix} \theta_{10} \\ \theta_{20} \\ \theta_{30} \end{bmatrix} + \begin{bmatrix} \theta_{11} \\ \theta_{21} \\ \theta_{31} \end{bmatrix} x \right)$$

¿Cómo escribimos una red neuronal profunda general?

- ▶ Podemos hacer lo anterior con todas las partes de la red y escribir todo en forma vectorial de la siguiente manera:
 - ▷ $\mathbf{h} = a(\theta_0 + \theta \mathbf{x})$
 - ▷ $\mathbf{h}' = a(\psi_0 + \Psi \mathbf{h})$
 - ▷ $y = \phi'_0 + \phi' \mathbf{h}'$
- ▶ Entonces podemos escribir todas las capas en términos de un vector de bias y una matriz de pesos
 - ▷ $\mathbf{h}_1 = a(\beta_0 + \Omega_0 \mathbf{x})$
 - ▷ $\mathbf{h}_2 = a(\beta_1 + \Omega_1 \mathbf{h}_1)$
 - ▷ $y = \beta_2 + \Omega_2 \mathbf{h}_2$

Hiperparámetros

- ▶ **¿Qué es un hiperparámetro?**

Parámetro que se define **antes** del entrenamiento (no se aprende con backprop).

- ▶ En redes profundas típicamente ajustamos:

- ▷ **K capas:** profundidad de la red.

- ▷ D_k **unidades por capa k :** largo de cada capa.

- ▶ Cada combinación define una familia de funciones; luego entrenamos (optimización de hiperparámetros).

Redes superficiales vs profundas

Propiedad	Superficiales	Profundas
Teorema de Aproximación	✓	✓
Regiones lineales	$D + 1$	$\max (D + 1)^K$
Facilidad de entrenamiento	Difícil	“Fácil”
Generalización	Depende	Depende

- ▶ Una red superficial puede aproximar funciones muy complejas si es suficientemente ancha.
- ▶ La ventaja de la profundidad no es simplemente “puede representar más funciones”, sino que **puede representar ciertas funciones mucho más eficientemente** mediante composición.

Redes superficiales vs profundas: Teorema de aproximación universal

- ▶ Una red neuronal con **una sola capa oculta**, suficiente anchura y una activación adecuada puede aproximar arbitrariamente bien cualquier función continua en un dominio compacto.
- ▶ Por lo tanto, **la profundidad no es necesaria para aproximación universal**.
- ▶ Las redes profundas también son aproximadores universales.
- ▶ La ventaja de la profundidad no es simplemente poder representar funciones que una red superficial no puede, sino poder representar **ciertas funciones de forma mucho más eficiente**.

aproximación universal $\neq$ eficiencia representacional

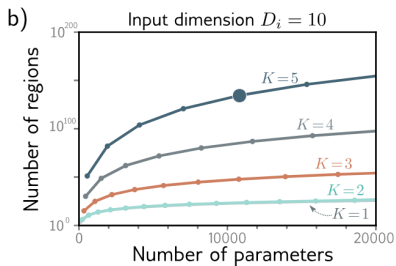
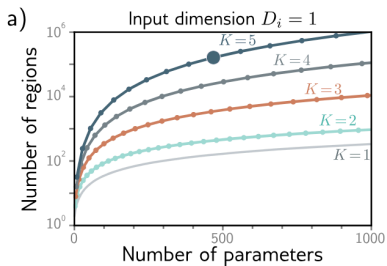
Redes superficiales vs profundas: número de regiones lineales

Para redes ReLU con **1 input y 1 output**:

- ▶ Una red superficial con D neuronas ocultas puede crear hasta: $D + 1$ regiones lineales.
- ▶ Una red profunda con K capas ocultas de $D > 2$ neuronas cada una puede crear hasta: $(D + 1)^K$ regiones lineales.
- ▶ El número de parámetros es: $3D + 1 + (K - 1)D(D + 1)$
- ▶ La profundidad puede hacer crecer el número de regiones **exponencialmente con** K , mientras que el número de parámetros crece aproximadamente de forma lineal con K .

profundidad $\Rightarrow$ mayor eficiencia representacional

Redes superficiales vs profundas: número de regiones lineales



Redes superficiales vs profundas: entrenamiento (próximamente)

- ▶ Aumentar la profundidad hace al modelo más expresivo, pero también puede hacer la optimización más difícil.
- ▶ En redes muy profundas pueden aparecer problemas como gradientes que desaparecen o explotan.
- ▶ Sin embargo, redes suficientemente sobreparametrizadas suelen ser sorprendentemente fáciles de optimizar con métodos basados en gradientes.
- ▶ Técnicas como buena inicialización, ReLU, normalización y conexiones residuales permiten entrenar redes con decenas o cientos de capas.

más profundidad $\nRightarrow$ entrenamiento más fácil

Redes superficiales vs profundas: generalización

- ▶ Una red más profunda **no necesariamente generaliza mejor** que una red superficial.
- ▶ En muchas tareas, las arquitecturas profundas logran un excelente desempeño porque pueden aprender representaciones jerárquicas y aprovechar grandes cantidades de datos.
- ▶ Redes muy sobreparametrizadas pueden alcanzar un error de entrenamiento cercano a cero e incluso memorizar etiquetas aleatorias.
- ▶ Aun así, en datos reales pueden generalizar sorprendentemente bien.

buena optimización $\neq$ buena generalización

Redes superficiales vs profundas: generalización

- ▶ La generalización depende de muchos factores: datos, arquitectura, optimización, regularización e inductive biases.
- ▶ Entender completamente por qué las redes profundas sobreparametrizadas generalizan tan bien sigue siendo un problema activo de investigación.